

Techniques implemented in this notebook

#	Technique	
1	BM25 Retrieval ✓	S
2	LLM Query Rewrite ✓	F
3	HyDE →	C
4	Dense Vector Retrieval	S
5	Reciprocal Rank Fusion (RRF)	N
6	Cross-Encoder Reranking }	F
7	Augmented Generation	F

✓ D3
✓ D1
✓ D4

✓ D3
✓ D5
✓ D6

find me all the
Document Tiller →
+ keyword match

XEAS →
Decomposition

TF-IDF

Query-LLM

Hybrid Retrieval
()

→ 10

→

LLM Query Rewrite
Decomposition
Expansion
LLM Rewrite
→ Query

Query → EMB → [✓]
Question

EMB → [800]

Cosine Similarity

→ D1 -
D2 -
D3 -

Top k=3

1 MULTIPLE RETRIEVALS

The same query is run against different retrievers (e.g., BM25, Vector, Hybrid).

Query "aircraft battery fire procedure"

Top=3

Retriever A (BM25)

Rank	Document (ID)	Score
1	D3	12.4
2	D7	9.1
3	D1	8.0
4	D9	7.2
5	D4	6.1
...	...	...

Retriever B (Vector)

Rank	Document (ID)	Score
1	D1	0.87
2	D3	0.82
3	D8	0.75
4	D4	0.72
5	D7	0.66
...	...	...

Retriever C (Hybrid)

Rank	Document (ID)	Score
1	D3	0.91
2	D1	0.88
3	D4	0.76
4	D5	0.71
5	D9	0.60
...	...	...

2 RECIPROCAL RANK SCORING

For each document, compute a score from each retriever using:

$$\text{RRF Score}_r(d) = \frac{1}{k + \text{rank}_r(d)}$$

where:

- r = retriever
- d = document
- $\text{rank}_r(d)$ = rank of document d in retriever r
- k = constant (commonly 60)

Documents not found in a retriever get score 0.

EXAMPLE (k = 60)

In Retriever A		In Retriever B		In Retriever C	
Rank	RRF Score	Rank	RRF Score	Rank	RRF Score
1	$1/(60+1) = 0.0164$	1	0.0164	1	0.0164
2	$1/(60+2) = 0.0161$	2	0.0161	2	0.0161
3	$1/(60+3) = 0.0159$	3	0.0159	3	0.0159
4	$1/(60+4) = 0.0156$	4	0.0156	4	0.0156
5	$1/(60+5) = 0.0154$	5	0.0154	5	0.0154
...	...	...	...	...	...

3 FUSE (SUM) THE SCORES

Add the RRF scores of each document across all retrievers.



Why RRF?

- Rewards documents that appear high in any retriever.
- Robust to score scale differences across retrievers.

Example: Combined RRF Score (k = 60)

Document	RRF Score from A	RRF Score from B	RRF Score from C	Sum (RRF Score)
D3	0.0164	0.0161	0.0164	0.0489
D1	0.0159	0.0164	0.0161	0.0484
D4	0.0154	0.0156	0.0159	0.0469
D7	0.0161	0.0154	0.0000	0.0315
D8	0.0000	0.0159	0.0000	0.0159
...	...	...	...	...

4 FINAL RANKING

Sort documents by the summed RRF score (highest first) to get the final ranking.



RESULT

Odd RRF produces a single, reliable, and balanced ranking by combining multiple retrieval perspectives.

Final Rank	Document (ID)	Combined RRF Score
1	D3	0.0489
2	D1	0.0484
3	D4	0.0469
4	D7	0.0315
5	D8	0.0159
...	...	...

sort

> LLM

1 REPRESENT AS VECTORS

Convert texts/items into numerical vectors using an embedding model.

Example
Query: "aircraft battery fire procedure"
Documents: D1, D2, D3, D4

Query Vector (q)

[0.20, 0.10, 0.40, 0.50, 0.10]

Document Vectors

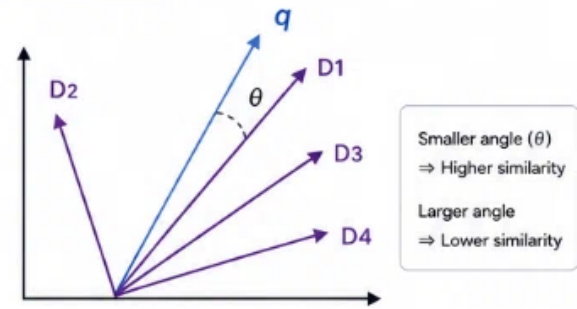
D1 [0.18, 0.12, 0.38, 0.48, 0.12]

D2 [0.05, 0.80, 0.10, 0.05, 0.05]

D3 [0.22, 0.08, 0.36, 0.50, 0.10]

D4 [0.10, 0.20, 0.30, 0.20, 0.40]

...



2 COSINE SIMILARITY FORMULA

The cosine similarity between query vector q and document vector d is:

$$\text{cosine}(q, d) = \frac{q \cdot d}{\|q\| \|d\|}$$

- $q \cdot d$ = dot product of q and d
- $\|q\|$ = magnitude (length) of q
- $\|d\|$ = magnitude (length) of d
- Range: -1 to 1
 - 1 → exactly the same direction (most similar)
 - 0 → perpendicular (no similarity)
 - 1 → opposite direction (least similar)

HOW IT'S COMPUTED (EXAMPLE)

For each document, compute cosine similarity with the query.

Document	Dot Product ($q \cdot d$)	$\ d\ $	Cosine Similarity ($q \cdot d / (\ q\ \ d\)$)
D1	0.468	0.657	0.989
D2	0.125	0.814	0.188
D3	0.470	0.678	0.994
D4	0.320	0.583	0.806
...			

($\|q\|$ for this example = 0.678)

3 RANK BY COSINE SIMILARITY

Sort documents by cosine similarity score in descending order.

- ★ Why Cosine Similarity?
- Captures orientation, not magnitude.
 - Works well for high-dimensional sparse data (e.g., text embeddings).
 - Widely used in Search, IR, NLP, Recommenders, and Clustering.

Rank	Document	Cosine Similarity	Interpretation
1	D3	0.994	Very similar
2	D1	0.989	Very similar
3	D4	0.806	Moderately similar
4	D2	0.188	Low similarity
...			

what is Tiller and what is Tractor

2 The BM25 Formula

$$\text{BM25}(q, d) = \sum_{t \in q} \text{IDF}(t) \cdot \frac{f(t, d) \cdot (k_1 + 1)}{f(t, d) + k_1 \cdot \left(1 - b + b \cdot \frac{|d|}{\text{avgdl}}\right)}$$

- Where:
- q = query, d = document
 - $f(t, d)$ = frequency of term t in document d
 - |d| = length of document d (number of words)
 - avgdl = average document length in the collection
 - IDF(t) = inverse document frequency of term t
 - k_1 (usually 1.2) = controls term frequency saturation
 - b (usually 0.75) = controls document length normalization

★ Key Ideas

- Higher term frequency (f) → higher score, but with diminishing returns.
- Rare terms (high IDF) → get higher weight.
- Very long documents are slightly penalized so they don't get an unfair advantage.

3 Working Example

Query: "aircraft battery fire"

Step 1: Check term frequency (f)

Count how many times each query term appears in each document.

Term frequencies in each document		
Document	Length	Term Frequency $f(t, d)$

appears in each document.

Step 2: Apply IDF weight

Rare terms (like "fire") get higher weight than common terms (like "aircraft").

Step 3: Normalize by document length

Slightly reduce the score for very long documents to keep the ranking fair.

Step 4: Sum scores for all query terms

Add the contribution of all query terms to get the final BM25 score for each document.

Document	(<i>l</i> _{<i>d</i>})	aircraft	battery	fire
D1	100	3	2	1
D2	120	1	1	0
D3	80	2	3	2
D4	200	2	1	5

Assume: IDF(aircraft)=1.0, IDF(battery) = 1.2, IDF(fire) = 1.5

avgdl = 125, $k_1 = 1.2$, $b = 0.75$

Final BM25 Scores			
Rank	Document	BM25 Score	Why it ranks here?
1	✓ D3 ✓	7.82	Contains all terms, high frequency
2	✓ D4	6.46	Strong match on rare term "fire"
3	✓ D1	6.12	Good match, but lower "fire" count
4	D2	2.83	Missing "fire", lower term frequency

Top k=3

Retrieval = KW → D3, D4, D1

Agent

Agentic AI

(

Software

↓

Tools, Guardrails

Perceive → Reason → Act

(find weather in chenna

(openweathermap - Tool

↳ Answer

Response

```
chain = rewrite_prompt | rewrite_llm | StrOutputParser()
```

Goal
↓
Understand requirements
↓
Search flights
↓
Compare prices
↓
Search hotels
↓
Check locations
↓
Build itinerary
↓
Notice a conflict

Agentic AI

↳ Broader concept

↓
↓

Break goal

↓
↳ let to do next

↳
Use tools

Observe

Accept Results

Towards

↳ Stop

↓
Change hotel
↓
Finalize plan

Agents

- ① Role → Senior Marketing Role
- ② focus → objective
- ③ Tools ✓
- ④ Co-ordination → A-A → Delegation → You do this
→ Hand-off →
- ⑤ Memory ↔ Run
- ⑥ Guardsails ✓↖

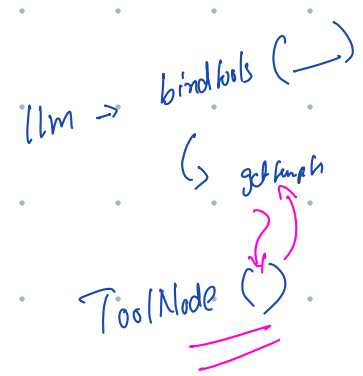
Tools



- Name
- Typedict
- Description → Docstring ""
- Decorator

Tool

```
@tool  
def get_temperature (str: city)  
    """  
    _____  
    _____  
    _____  
    _____  
    """  
    _____  
    _____  
    _____  
    return temp
```



Guardrails

Safe Boundary

→ Delete → HTML
Harmful
PII → LLM
NER

DPDP
(250 cases)

Output ← LLM ← Input
JSON User

Deterministic

→ 3 words
Harmful words
Toxic
Blocked / Blacklist
→ Explosive

Semantic → LLM
↳ Bomb Safe - Not Safe

Langchain → Modular framework

↳ Components

Langgraph → Stateful graph agent
Langsmith → Tracing & observability
Langserve → Deployment

Embedder
C A U
Con-LLM → Models

Vectostores
chromadb

Agent

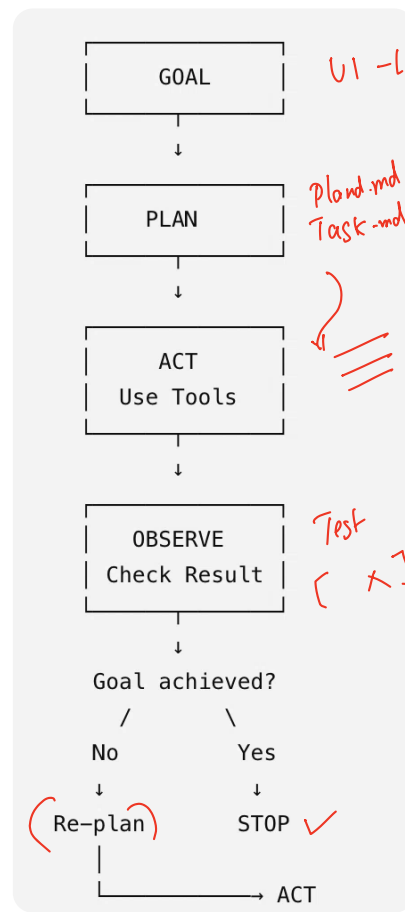
tools

text splitters
) (

Document Loaders
Pdf | csv | word

Output parser
↳ String output gen

Agentic Loop



UI - login

pland.md
Task.md

✓
✓
✓
✓

Test
[X]

(Re-plan)

STOP ✓

ACT

Agentic Loop